Design and Analysis of Algorithm

Lecture 19: Stack

Dr. Tauheed Ahmed

Content

- Introduction
- Operations on Stack

Stack

A stack is a list of elements in which an element may be inserted or deleted only at one end, called the top of the stack.

A Stack works on the LIFO process (Last In First Out), i.e., the element that was inserted last will be removed first.

To implement the Stack, it is required to maintain a pointer to the top of the Stack, which is the last element to be inserted because we can access the elements only on the top of the Stack.

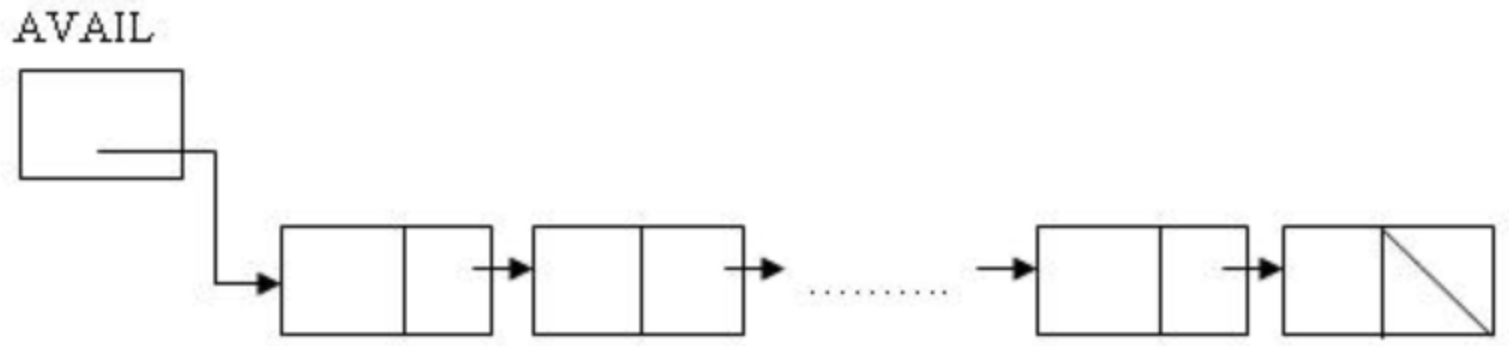
Stack: Basic Idea

Stacks store elements in a sequential order, where each element has a predecessor and a successor, except for the first and last elements. It is named stack as it behaves like a real-world stack, ***for example – a deck of cards or a pile of plates, etc.***

Stack: Application

Avail List

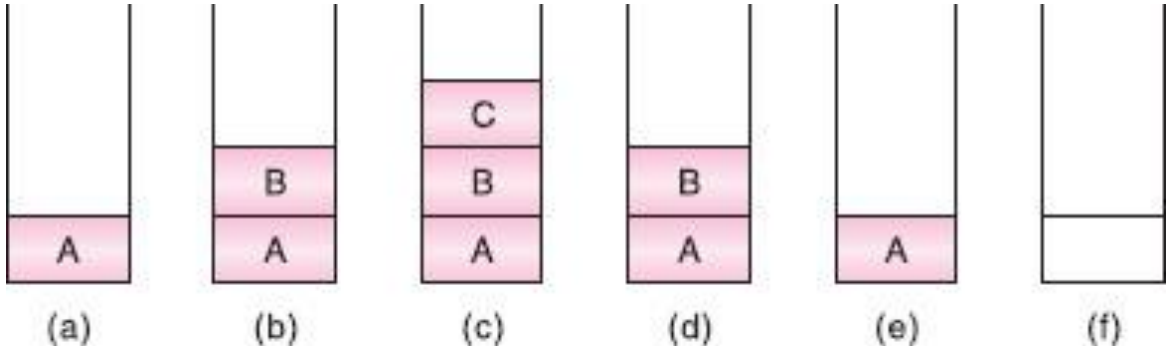
Recall that free nodes were removed only from the beginning of the AVAIL list, and that new available nodes were inserted only at the beginning of the AVAIL list.



Stack: Application

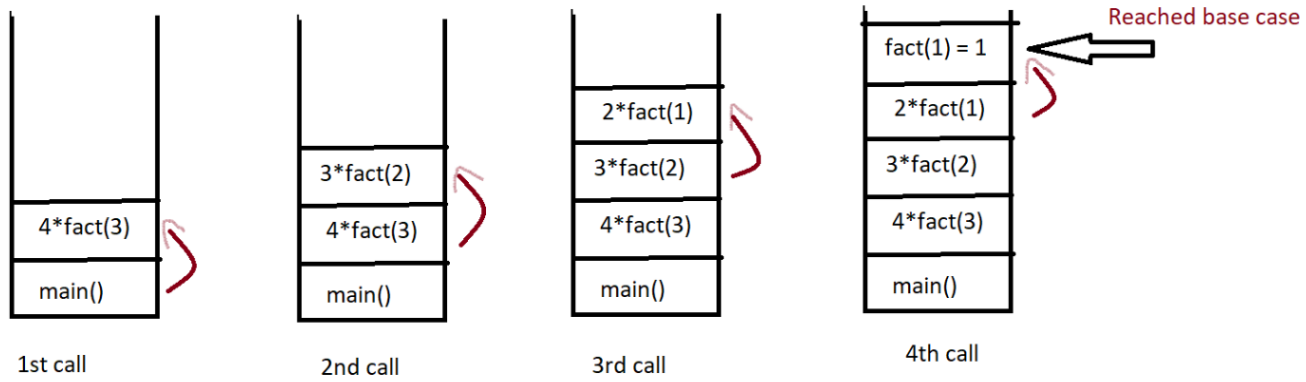
Postponed Decisions

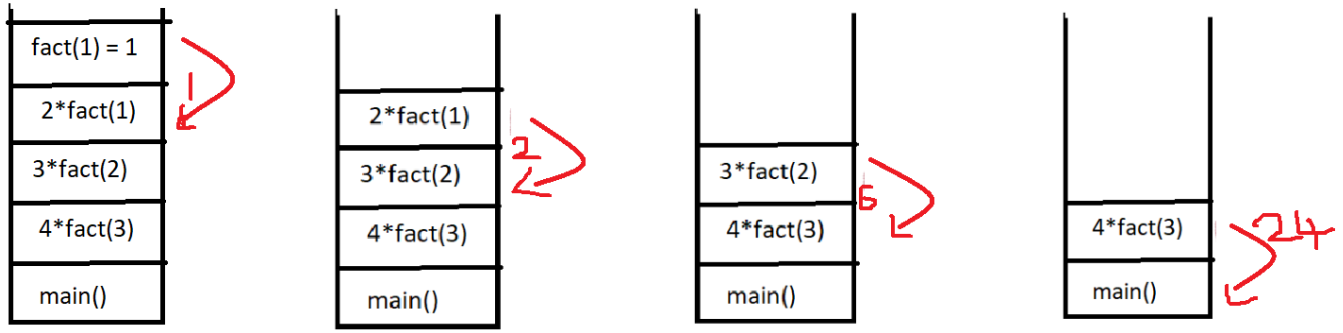
Stacks are frequently used to indicate the order of the processing of data when certain steps of the processing must be postponed until other conditions are fulfilled.



Stack: Application

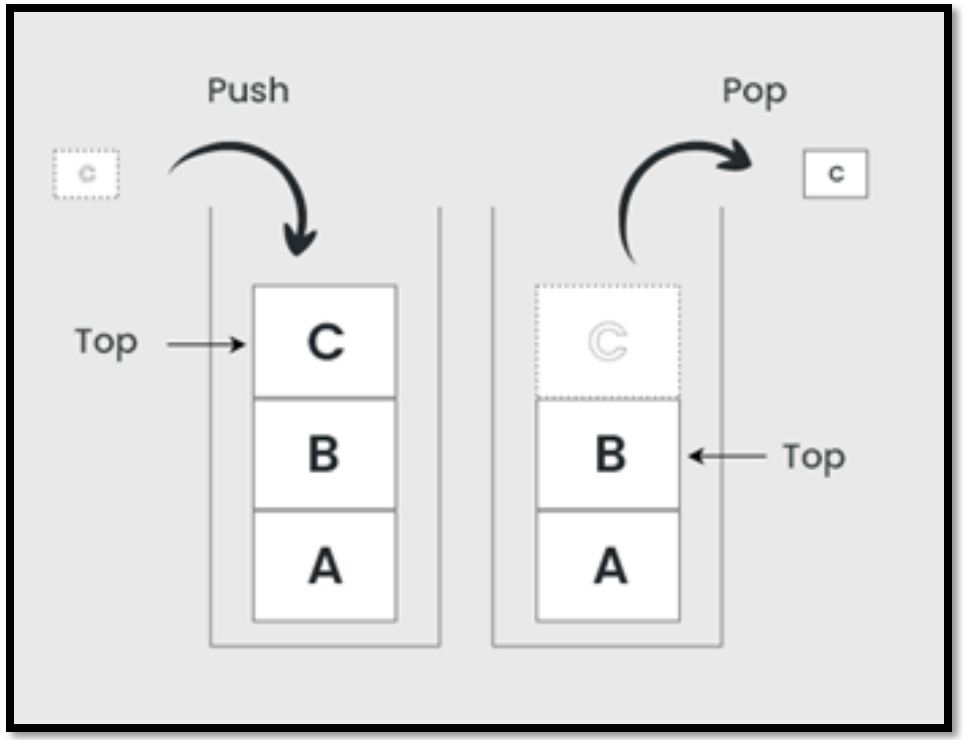
Recursion





Returning function calls

Stack: Representation



- Can be implemented by means of Array, or Linked List.
- Stack can either be a fixed size or dynamic.

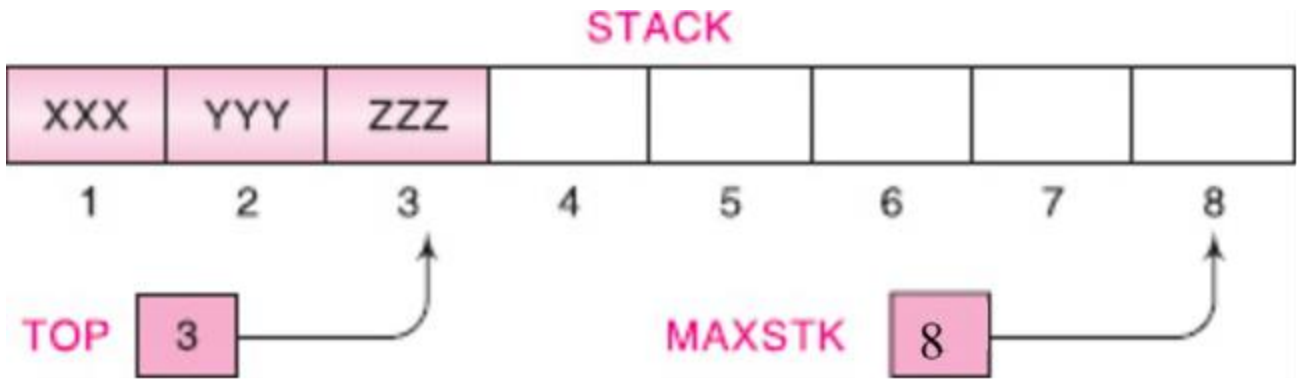
Stack: Operations

The primary operations on a stack, a Last-In, First-Out (LIFO) data structure, are

Push:	• Adds a new element to the top of the stack.
Pop:	• Removes the element from the top of the stack.
Peek:	• Returns the element at the top of the stack without removing it.
isEmpty:	• Checks if the stack is empty.
isFull:	• Checks if the stack is full (if applicable, for stacks with a fixed size).
size:	• Returns the number of elements in the stack.

Stack : Array Representation

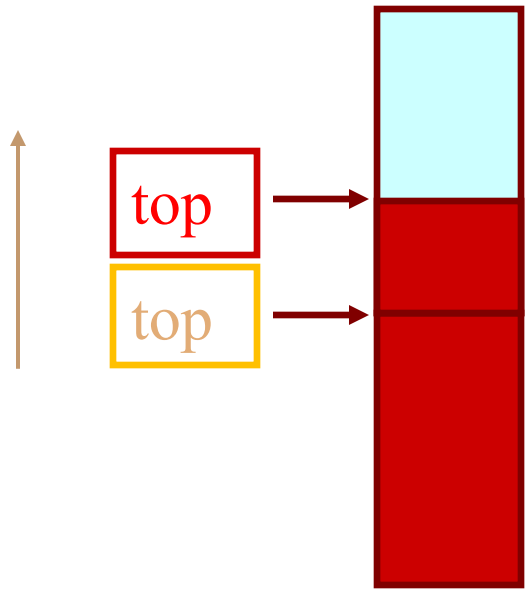
Stacks will be maintained by a linear array STACK; a pointer variable TOP, which contains the location of the top element of the stack; and a variable MAXSTK which gives the maximum number of elements that can be held by the stack.



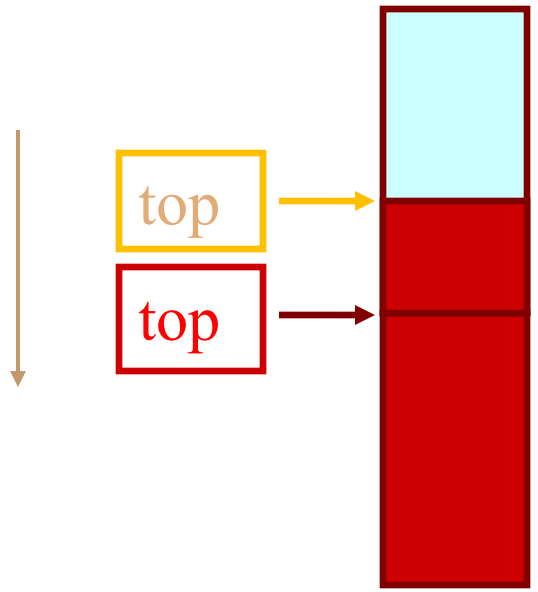
Since TOP = 3, the stack has three elements, XXX, YYY and ZZZ; and since MAXSTK = 8, there is room for 5 more items in the stack.

Stack : Push and Pop Operation

Array Implementation

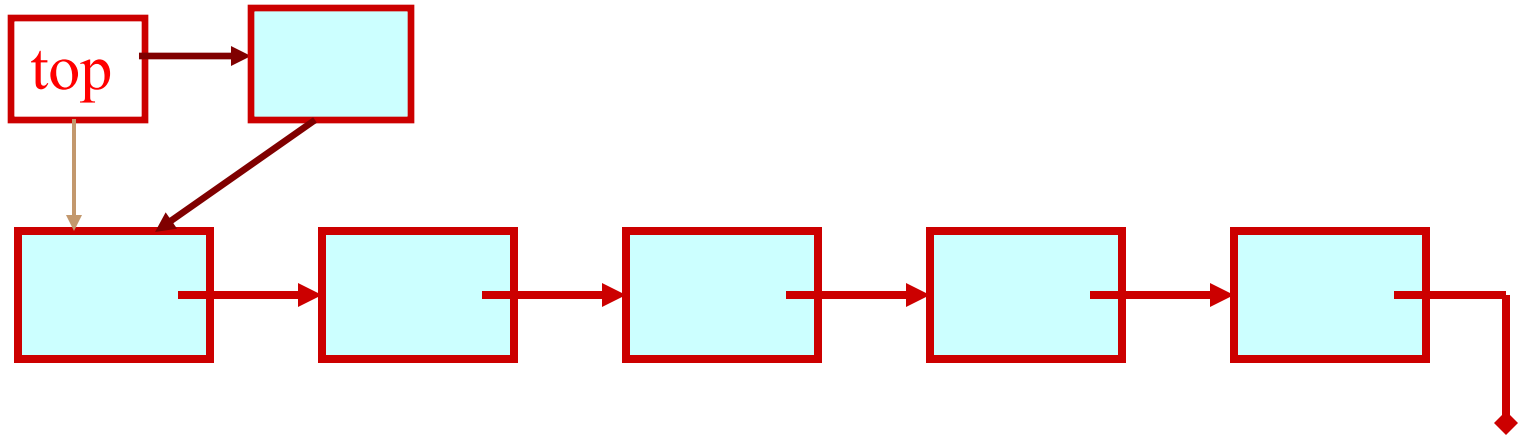


PUSH

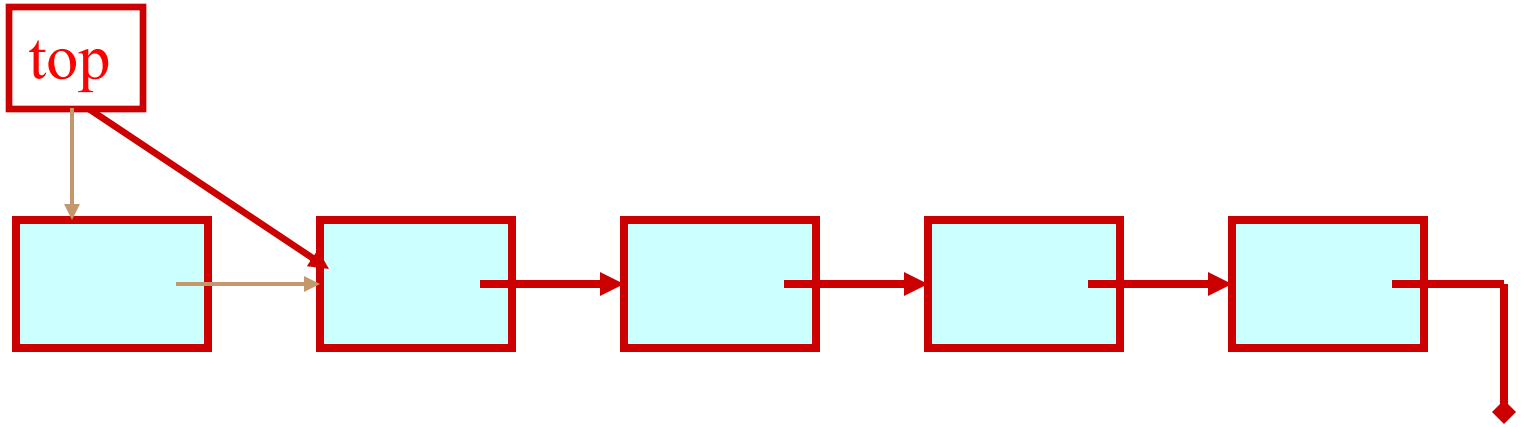


POP

Stack : Push and Pop Operation



Linked List Implementation



Stack : Push and Pop Operation

- In the array implementation, we would:
  - Declare an array of fixed size (which determines the maximum size of the stack).
  - Keep a variable which always points to the “top” of the stack.
    - Contains the array index of the “top” element.
- In the linked list implementation, we would:
  - Maintain the stack as a linked list.
  - A pointer variable top points to the start of the list.
  - The first element of the linked list is considered as the stack top.

Stack : Declaration

Array

```
#define MAX 100

typedef struct Stack {
    int arr[MAX];
    int top;
} Stack;
```

Linked List

```
typedef struct Stack {
    int data;
    struct Node* next;
} Stack;
```

Stack : Creation

Array

```
void create (stack *s)
{
   s->top = -1;

   /* s->top points to
      last element
      pushed in;
      initially -1 */
}
```

Linked List

```
void create (stack **top)
{
   *top = NULL;

   /* top points to NULL,
      indicating empty
      stack              */
}
```

Stack : Push Operation

Array

```
void push (stack *s, int element)
  {
   if (s->top == (MAXSIZE-1))
   {
     printf ("\n Stack overflow");
         exit(-1);
     }
     else
     {
         s->top++;
         s->arr[s->top] = element;
     }
  }
```

Linked List

```
void push (stack **top, int element)
{
    stack *new;

    new = (stack *)malloc (sizeof(stack));
    if (new == NULL)
    {
       printf ("\n Stack is full");
       exit(-1);
    }

    new->data = element;
    new->next = *top;
    *top = new;
}
```

Stack : Pop Operation

Array

```
int pop (stack *s)
  {
     if (s->top == -1)
     {
        printf ("\n Stack underflow");
        exit(-1);
     }
     else
     {
        return (s->st[s->top--]);
     }
  }
```

Linked List

```
int pop (stack **top)
{
   int t;
   stack *p;
   if (*top == NULL)
   {
      printf ("\n Stack is empty");
      exit(-1);
   }
   else
   {
      t = (*top)->value;
      p = *top;
      *top = (*top)->next;
      free (p);
      return t;
   }
}
```

Stack : IsEmpty Operation

Array

```
int isempty (stack *s)
{
   if (s->top == -1)
          return 1;
   else
          return (0);
}
```

Linked List

```
int isempty (stack *top)
{
   if (top == NULL)
         return (1);
    else
         return (0);
}
```

Stack : Application

- Page-visited history in a Web browser
- Undo sequence in a text editor
- Infix to postfix conversion
- Postfix expression evaluation
- Recursion implementation

Arithmetic Notation

Let Q be an arithmetic expression involving constants and operations. The binary operations in Q can have varying levels of precedence. Specifically, we consider three distinct precedence levels for the five common binary operations.

Highest:	Next highest:	Lowest:
• Exponentiation (↑)	• Multiplication (*) and division (/)	• Addition (+) and subtraction (–)

Arithmetic Notation: Example

What is the result of the following arithmetic expression:

$$2 \uparrow 3 + 5 * 2 \uparrow 2 - 12 / 6$$

First evaluate operators with highest precedence

$$8 + 5 * 4 - 12 / 6$$

Next highest precedence

$$8 + 20 - 2$$

Lowest precedence

Arithmetic Notation: Infix

In our previous example the operator symbol is placed between its two operands. This is called infix notation.

Example

$$A + B,\ C - D,\ E * F,\ G/H$$

In this representation we must distinguish between following expressions :

$$(A + B) * C \ and \ A + (B * C)$$

It can be done either by using parenthesis. Thus, the order of the operators and operands in an arithmetic expression does not uniquely determine the order in which the operations are to be performed.

Arithmetic Notation: Polish

It refers to the notation in which the operator symbol is placed before its two operands

Example

$$+ A\ B, -C\ D, \ *E\ F, \ /GH$$

The fundamental property of Polish notation are:

- The order in which the operations are to be performed is completely determined by the positions of the operators and operands in the expression.
- Accordingly, one never needs parentheses when writing expressions in Polish notation.

Arithmetic Notation: Reverse Polish

It refers to the notation in which the operator symbol is placed after its two operands:

Example

$$A\ B+,\ C\ D-,\ \ E\ F*,\ \ GH/$$

One never needs parentheses to determine the order of the operations in any arithmetic expression written in reverse Polish notation.

Arithmetic Notation: Evaluation

The computer usually evaluates an arithmetic expression written in ***infix*** notation in two steps.

- Converts the expression to postfix notation,
- Evaluates the postfix expression.

In each step, the stack is the main tool that is used to accomplish the given task.

Arithmetic Notation: Transformation

Algorithm

1. Print operands as they arrive.
2. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
3. If the incoming symbol is a left parenthesis, push it on the stack.
4. If the incoming symbol is a right parenthesis, pop the stack and print the operators until you see a left parenthesis. Discard the pair of parentheses.
5. If the incoming symbol has higher precedence than the top of the stack, push it on the stack.
6. If the incoming symbol has equal precedence with the top of the stack, use association. If the association is left to right, pop and print the top of the stack and then push the incoming operator. If the association is right to left, push the incoming operator.
7. If the incoming symbol has lower precedence than the symbol on the top of the stack, pop the stack and print the top operator. Then test the incoming operator against the new top of stack.
8. At the end of the expression, pop and print all operators on the stack. (No parentheses should remain.)